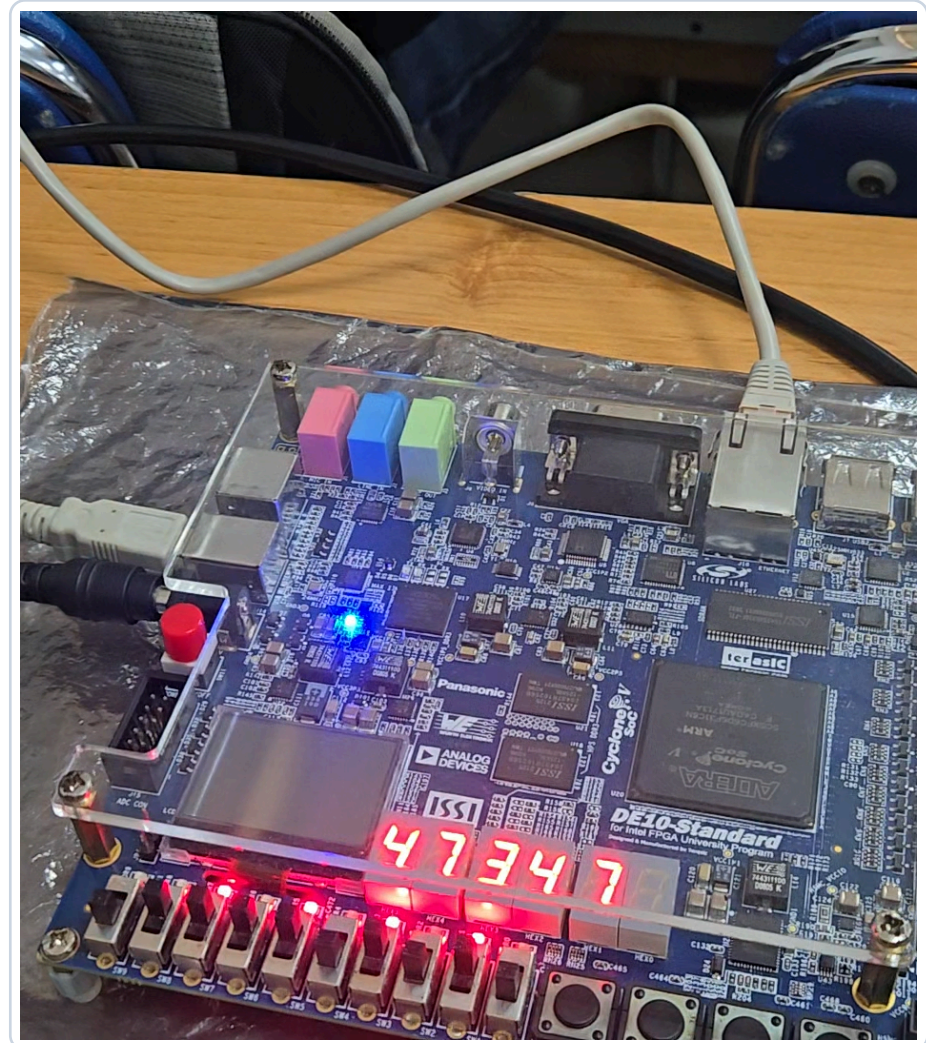


System-integrated SoC Ethernet: firmware and PC TCP command application

DE10-Standard Cyclone V SoC FPGA

HPS Linux -> TCP -> Lightweight HPS-FPGA Bridge -
> HEX0..HEX5

Report Information



Instructors M.Sc. Tran Tuan Kiet; M.Sc. Do Quoc Minh Dang; M.Sc. Nguyen Nhu Hoang
Van Dinh Nam; Luong Hai

Members Long; Tran Si Nam; Le Tan Phi Pha; Vu Chau Thang Loi

Duration 10-15 minutes

Main Message

Do not present isolated modules.

The presentation follows an end-to-end control path that can be verified:

Engineering Flow

PC GUI -> **TCP** -> **HPS Linux** -> **Bridge** -
> **HEX**

1

end-to-end control path

6

maximum characters on
HEX0..HEX5

5000

board TCP service port

LW

bridge in the 0xFF200000
region

Main point for reviewers: the team did not only build a GUI. The software path reaches a hardware peripheral that can be observed directly.

Problem and Scope

Input

- User enters the board IP, TCP port and a payload up to 6 characters.
- Client sends the payload through a TCP socket inside the same LAN.
- Board returns OK:<text> or ERR:<reason>.

Output

- Normalized payload is displayed on HEX5..HEX0.

Deliberate Boundaries

- Local LAN only, without Internet or NAT traversal.
- Function correctness first; latency and throughput are not measured quantitatively.
- Protocol is currently a simple text line, without a len/cmd/crc frame.
- devmem fits the prototype; no kernel driver is implemented.

Prototype Value

Problem and Scope

- PC-side log is available for request/response checking.
- Android client uses the same service to verify an independent endpoint.

The team has observable evidence at each layer: Linux shell, eth0 IP, TCP socket, OK/ERR response and physical HEX state.

Overall Architecture

PC GUI

Enter payload,
send TCP and
write log

TCP

STREAM socket
with one-connec-
tion request/re-
sponse

HPS Linux

Python server
listens on
0.0.0.0:5000

Sanitize

Uppercase, filter
characters, trim or
pad to 6

Bridge

devmem writes PIO
registers

HEX

HEX5..HEX0 dis-
plays left to right

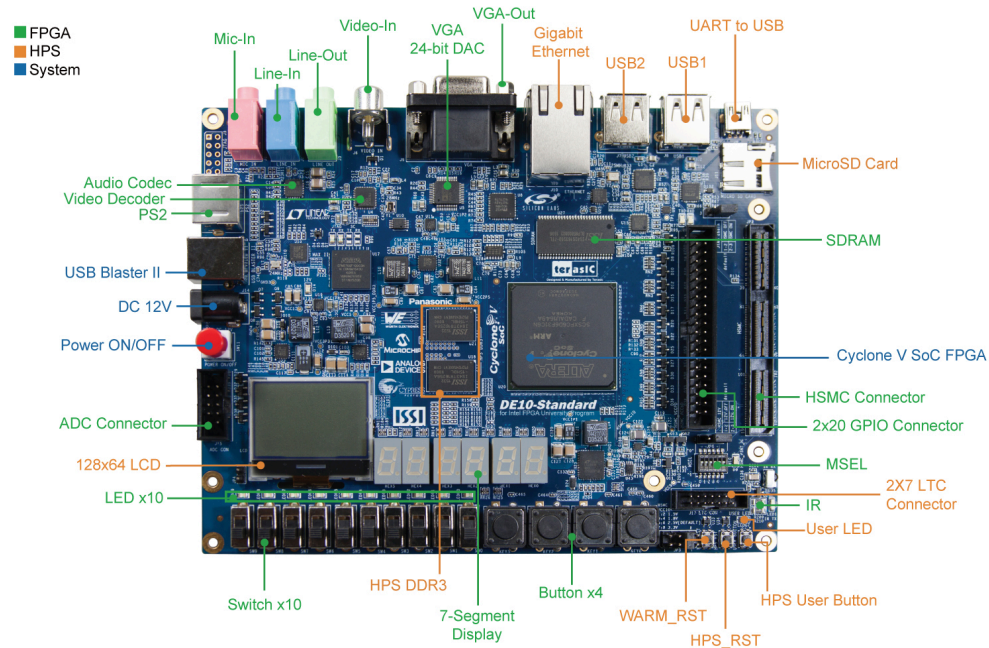
Design Reasoning

Each layer has its own input, output and test method. When a fault appears, the team isolates the first broken layer instead of guessing across the whole system.

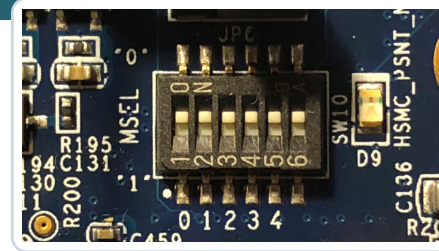
Key Speaking Point

The Android client is not decoration. It proves that the board TCP service is not locked to the PC GUI endpoint.

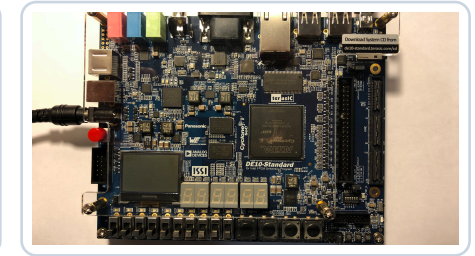
Hardware Platform and Bring-up



DE10-Standard Cyclone V SoC FPGA



Ethernet LAN



USB-UART + Ethernet + power

Minimum Bring-up

1. Load/configure the bitstream and boot files.

Hardware Platform and Bring-up

2. Boot Linux from MicroSD and access the shell over USB-UART.
3. Enable `eth0` and request an IP address through DHCP.
4. Run the TCP server and test from a PC inside the LAN.

Platform Designer and Address Map

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
☒		clk_0	Clock Source	clk	exported					
		clk_in	Clock Input	reset	clk_0					
		clk_in_reset	Reset Input	reset	clk_0					
		clk	Clock Output	reset	clk_0					
		clk_reset	Reset Output	reset	clk_0					
☒		hps_0	Arria V/Cyclone V Hard Proce...	hps_0_ddr						
		memory	Conduit	hps_0_io						
		hps_io	Conduit	hps_0_io						
		h2f_reset	Reset Output	hps_0_io						
		f2h_sdr0_clock	Clock Input	hps_0_io						
		f2h_sdr0_data	Avalon Memory Mapped Slave	hps_0_io						
		f2h_sdr1_clock	Clock Input	hps_0_io						
		f2h_sdr1_data	Avalon Memory Mapped Slave	hps_0_io						
		h2f_lw_axi_clock	Clock Input	hps_0_io						
		h2f_lw_axi_master	AXI Master	hps_0_io						
☒		pio_0	PIO (Parallel I/O) Intel FPGA IP	pio_0						
		clk	Clock Input	pio_0						
		reset	Reset Input	pio_0						
		s1	Avalon Memory Mapped Slave	pio_0						
		external_connec...	Conduit	pio_0						
☒		msgdma_0	Modular Scatter-Gather DMA I...	msgdma_0						
		mm_read	Avalon Memory Mapped Master	msgdma_0						
		mm_write	Avalon Memory Mapped Master	msgdma_0						
		clock	Clock Input	msgdma_0						
		reset_n	Reset Input	msgdma_0						
		csr	Avalon Memory Mapped Slave	msgdma_0						
		descriptor_slave	Avalon Memory Mapped Slave	msgdma_0						
		csr_irq	Interrupt Sender	msgdma_0						
☒		pll_0	PLL Intel FPGA IP	pll_0						
		refclk	Clock Input	pll_0						
		reset	Reset Input	pll_0						
		outclk0	Clock Output	pll_0						

Platform Designer system

Hardware-Software Contract

pio_hex0 0xFF200040 HEX0

pio_hex1 0xFF200050 HEX1

pio_hex2 0xFF200060 HEX2

pio_hex3 0xFF200070 HEX3

pio_hex4 0xFF200080 HEX4

pio_hex5 0xFF200090 HEX5

Quartus Result

Platform Designer and Address Map

The top-level VHDL connects `pio_hex0_external_export` through `pio_hex5_external_export` to `HEX0` through `HEX5`. HPS code only needs to write the agreed PIO addresses.

Fitter successful. Logic utilization: **1,960 / 41,910 ALMs (5%)**. Total registers: **2,614**. Total pins: **302 / 499 (61%)**.

Firmware on HPS Linux

handle_client()

Receives `recv(1024)`, decodes UTF-8, handles one request and closes the connection.

sanitize_text()

Strips, uppercases, filters unsupported characters, then trims or pads to exactly 6 characters.

seg() + write_hex()

Encodes active-low seven-segment values and calls `devmem` to write 6 PIO registers.

Active-low

Server Core Logic

```
s.bind(("0.0.0.0", 5000))
s.listen(5)

data = conn.recv(1024)
text = sanitize_text(data)

v5, v4, v3, v2, v1, v0 = map(seg, text[:6])
devmem 0xFF200040..0xFF200090 32 value

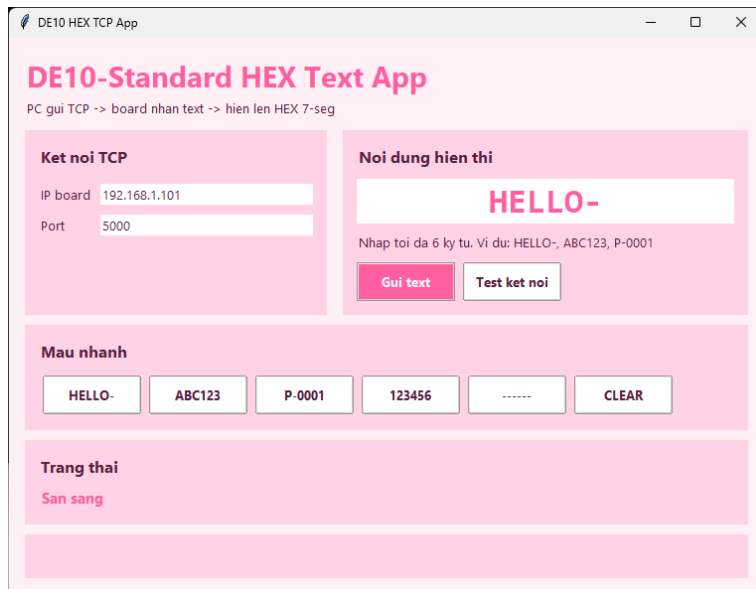
conn.sendall(("OK:%s\n" % text).encode())
```

If a reviewer asks about errors: the server returns `ERR:<reason>`, which separates socket errors, `devmem` errors, root-permission issues and wrong address-map issues.

Firmware on HPS Linux

0 means segment on, and 1 means segment off. Therefore 8 = 0, while blank space is 127.

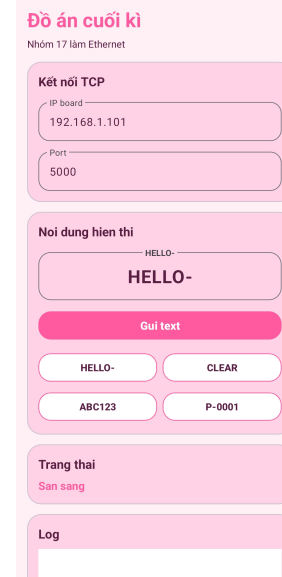
Client Applications



PC GUI with Python/Tkinter

PC GUI

- Separate thread for TCP actions so the interface does not freeze.



Android client using the same TCP service

Android client

- Sends the same newline-terminated UTF-8 payload.

Client Applications

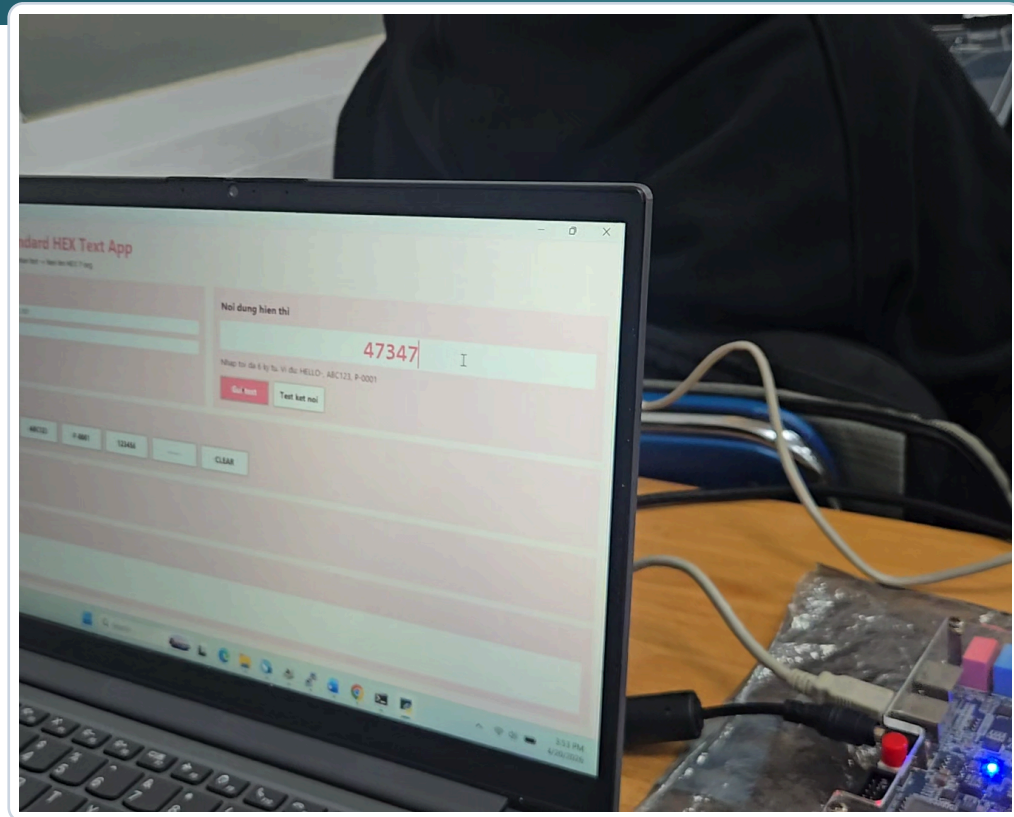
- 5-second socket timeout.
- Clear log for sent line, received line and error.

- Receives the same `OK:<text>` or `ERR:<reason>`.
- Used to verify that the server does not depend on one endpoint.

End-to-End Testing

Five-Step Procedure

1. Boot Linux and access the shell through USB-UART.
2. Enable eth0, request a DHCP IP and ping to check the network.
3. Run the TCP server on port 5000.
4. Send a payload from the PC GUI and receive OK:<text>.
5. Compare HEX5..HEX0 with the normalized string.



Laptop - router - board test environment

Test Payloads

End-to-End Testing

HELL0-, ABC123, P-0001, 123456,
-----, empty string, strings longer
than 6 characters and unsupported
characters.

Results and Fault Isolation



PC and Android controlling the same TCP service

Functional Result

Fault Isolation Tree

- No shell: check power, MSEL, MicroSD and USB-UART.
- Shell exists but no IP: check Ethernet, DHCP and udhcpd.
- IP exists but client cannot connect: check the server and port 5000.
- OK response but wrong HEX output: check `seg()`, `HEX5` . `HEX0` order and VHDL.
- ERR response: read the reason, then check `devmem`, root permission and PIO address.

Results and Fault Isolation

PC/Android -> TCP -> HPS Linux -> PIO
-> HEX works repeatedly inside LAN.
Valid payloads receive OK.

This is the strongest point: the team has a layer-by-layer debugging method, not only a one-time demo.

Limitations and Future Work

Current Limitations

- Server does not auto-start after re-boot.
- Protocol is still a simple text line.
- devmem fits the prototype but is not optimal for a long-term product.
- No authentication, encryption or protection against unintended commands.
- Latency and throughput are not measured quantitatively.

Next Development Steps

- Auto-start the server with init or systemd.
- Design a len/cmd/payload/crc frame.
- Replace devmem with mmap or a kernel driver.
- Add simple LAN authentication.
- Add automated test scripts for multiple payloads.

Limitations and Future Work

Answer When Asked Deeply

These limitations do not reduce the project value because the course goal is to integrate and verify the hardware-software chain. The team knows which parts are prototype-level and which parts would need upgrading for a more serious system.

Conclusion

The team built a SoC Ethernet prototype with a clear end-to-end control path that can be tested and explained layer by layer.

Main takeaway: PC/Android sends TCP commands, HPS Linux processes and writes through the Lightweight HPS-FPGA Bridge, and FPGA fabric displays the result on HEX0..HEX5.

Thank you for listening.